Input Function in C

Input Function

scanf() Function

- When the C program needs to **get input from the user**, we should use **scanf** function.
- To get input from the user, we should instruct the scanf function the following details,

    1. **Specify the format** i.e. which data type we are going to get from the user.
    2. Give the **variable address** where we want to store the value.

Syntax:

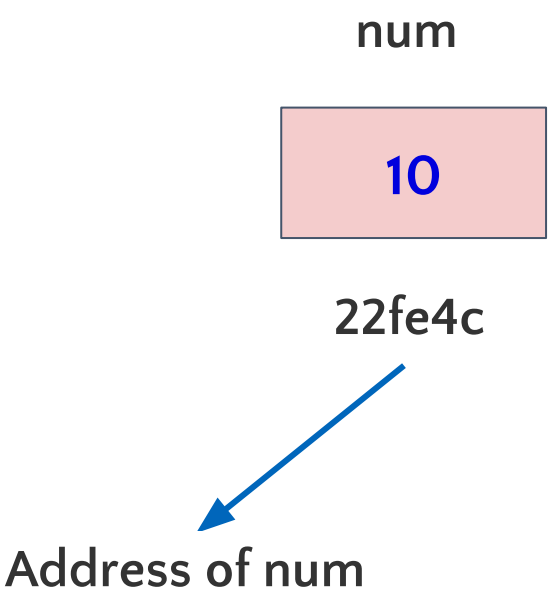
```
scanf("<format specifier>" , <address of variable>);
```

Contd...

How to get the address of a variable?

- Using **ampersand (&)** operator, we can get the address of a variable.
- In C, we will refer **&** as **address of operator.**
- In scanf, we must give the **address of a variable** not variable name.

```
int num = 10;
```



num = 10

&num = 22fe4c

How to Store Input in Variables

- To store input in a variable using `scanf`(), you need to pass the memory address of the variable as an argument to the function using the `&` (**address of**) operator.
- This is because `scanf`() expects an address as arguments to store input values directly in memory locations.
- Example of using `scanf`() to read an integer value from the user and store it in a variable called `num`:

```
int num;
printf("Enter an integer: ");
scanf("%d", &num);
```

- In this example, the `%d` format specifier tells `scanf`() to expect an integer input value.
- The memory address of the `num` variable is passed to `scanf`() using the `&` operator, which returns a address of the memory location of num.

Sample Program

```
#include<stdio.h>

int main()
{
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    printf("\The numbers is: %d",num);
    return 0;
}
```

Contd…

```c
//Program in C to enter the age and weight of a person

#include<stdio.h>

int main()
{
   int   age, weight;

   printf("Enter your age\n");
   scanf("%d", &age);
   printf("Enter your weight\n");
   scanf("%d", &weight);
   printf("age = %d \n",age);
   printf("weight = %d \n",weight);
   return 0;
}
```

Contd…

Sample program

- This program will get values from the user and print it.

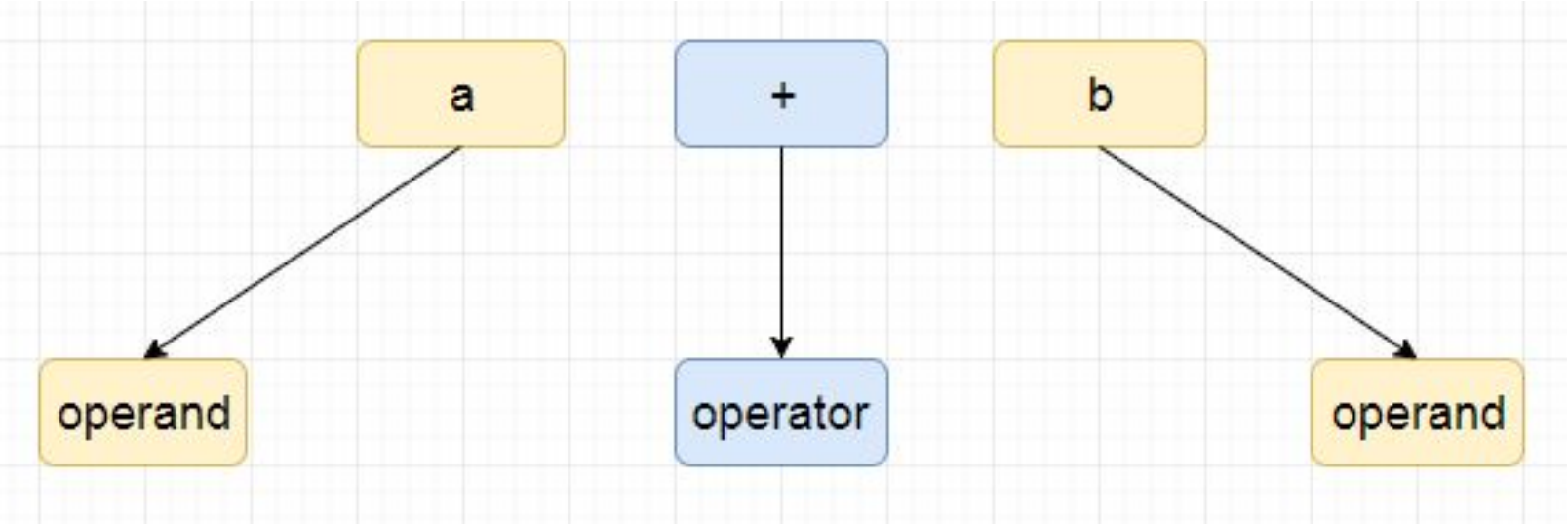
```
#include<stdio.h>

int main()
{
      int   age, weight;

      printf(“Enter your age and weight”);

      //getting input from the user and storing it into memory addresses
      scanf(" %d%d ", &age, &weight);

      printf("\nage = %d \nweight = %d \n",age,weight);

      return 0;
}
```

Operators in C

1. Operators in C

- Operators are used to perform mathematical and logical calculation in program.
- The tokens (or symbols) that trigger some kind of operations are called ***operators***.
- An operation requires an ***operator*** and ***operand***.
- ***Operands*** are the data on which the specified task is carried out.
- ***Operators*** are used in programs to manipulate data and variables,

Operator vs Operand



Contd…

- C operators can be classified into a number of categories. However, there is at general classification ***based on the number of operands required for an operation***.

Classification of Operators

1. Unary Operator

- Only **one operand** is required to perform calculation.

2. Binary Operator

- **Two operands** are required to perform calculation.

3. Ternary Operator

- **Three operands** are required to perform calculation.

1.1 Arithmetic Operators

- Arithmetic operators are used to perform basic mathematical calculations.
  Addition (**+**), subtraction (**-**), multiplication (*****), division (**/**), modulo operations (**%**).
- Except the **%** (**modulus operator**), all others work with both **integer** and **floating** point data.
- The **%** operator works only on **integer** operands.

Operators	Meaning	Example	Result
+	Addition	4+2	6
-	Subtraction	4-2	2
*	Multiplication	4*2	8
/	Division	4/2	2
%	Modulus operator to get remainder in integer division	5%2	1

Example: (a=100,b=10,ans=0)

ans = a + b; ans will be 110.

ans = a - b; ans will be 90.

ans = a * b; ans will be 1000.

ans = a / b; ans will be 10.

ans = a % b; ans will be 0.
Because 10 will perfectly divide 100 so remainder will be zero.
8 % 3 = 2.
5 % 4 = 1.

Sample Program

```
#include<stdio.h>
int main()
{
    int num1, num2;
   printf(“Enter 2 numbers”);
   scanf(“%d%d”,&num1, &num2);

   printf("num1 + num2 = %d\n",num1 + num2);
   return 0;
}
```

Sample Program

```c
// Program in C to perform arithmetic operations

#include<stdio.h>
int main()
{
   int num1, num2;
   printf("Enter 2 numbers");
   scanf("%d%d",&num1, &num2);
    printf("num1 + num2 = %d\n",num1 + num2);
    printf("num1 - num2 = %d\n",num1 - num2);
    printf("num1 * num2 = %d\n",num1 * num2);
    printf("num1 / num2 = %d\n",num1 / num2);
    printf("num1 modulo num2 = %d\n",num1 % num2);

    return 0;
}
```

WAP in C to get the user input & find the area & perimeter of a square

```c
#include<stdio.h>
int main()
{
    int side;
    printf("Enter the side of the square");
    scanf("%d", &side);
    printf("The area = %d \n", side*side);
    printf("The perimeter=%d \n", 4*side);
    return 0;
}
```

```c
#include<stdio.h>
int main()
{
    int side, area, perimeter;
    printf("Enter the side of the square \n");
    scanf("%d", &side);
    area = side*side;
    perimeter = 4 * side;
    printf("The area = %d \n", area);
    printf("The perimeter=%d \n",perimeter);
    return 0;
}
```

1.2 Assignment Operator

- Using assignment operators, we can assign value to the variables.
- Equality sign (=) is used as an assignment operator in C.

Example

```
int var = 5;     //Here, value 5 has assigned to the variable var.
```

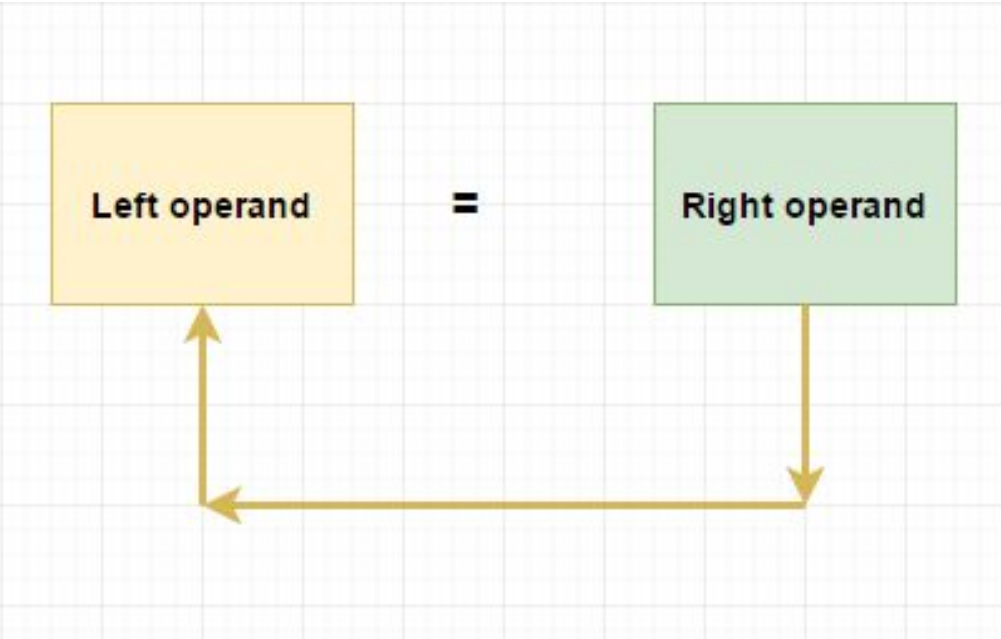
Program

```
int main()
{
    int a = 10, b;
    b = a;

    printf("a = %d \t b = %d\n",a,b);

    return 0;
}
```

- Here, value of a has assigned to the variable b. Now, both a and b will hold value **10**.

Contd…

- Basically, the value of **right-side operand** will be assigned to the **left side operand**.




```
#include<stdio.h>
int main()
{
    int a,b;
    printf("Enter 2 integer numbers");
        scanf("%d%d",&a,&b);
    a+=b; // same as a=a+b
    printf("value of a+b = %d\n",a);
    // a will hold the result
    return 0;
}
```

WAP in C to enter 2 numbers as user input and find its **quotient** and **remainder** and display it.

```c
#include<stdio.h>
int main()
{
    int a,b;
    printf("Enter 2 integers \n");
    scanf("%d%d",&a,&b);
    printf("The quotient = %d, Remainder = %d \n", a/b, a%b);
    return 0;
}
```

Contd…

Compound Assignment Operators

Operator	Meaning	Example (a = 10 , b = 5)
+=	L=L+R add left and right operand and assign result in left	**a+=b;**same as a=a+b after execution **a** will hold 15
-=	L=L-R subtract right operand from left operand and assign result in left	**a-=b;**same as a=a-b after execution **a** will hold 5
***=**	L=L*R multiply both right and left operand and store result in left	**a*=b;**same as a=a*b after execution **a** will hold 50
/=	L=L/R divides left operand by right operand and store result in left	**a/=b;**same as a=a/b after execution **a** will hold 2
%=	L=L%R After left and right operand division, the remainder will be stored in left	**a%=b;**same as a=a%b after execution **a** will hold 0

1.3 Increment and Decrement Operator

- C provides two special arithmetic operators, namely

  Increment operators (++) and **Decrement operators (--)** .
- Both these are **unary** operators and the operand should be a numeric (int, float or char) variable.
- The **increment operator** (++) **adds 1** to its operand and the **decrement operator** (--) **subtracts 1** from its operand.
- The result will be stored in the same operand and that is why it should be a variable.

  Example:

```
int a = 5, b = 3;

a++;    //After this statement execution, the value of a will be 6.

b--;    //After this statement execution, the value of b will be 2.
```

The values of a and b will be **6** and **2** respectively after the execution of these statements.

i.e.; **a++;** is equivalent to **a = a+1;** and **b--;** is same as **b = b-1;**

Contd…

- There are two versions of these operators: **postfix** and **prefix**.
- In **postfix notation**, the operator is placed after the operand as in **a++** and **b--;**
- but in **prefix notation**, the operator precedes the operand as in **++a** and **--b.**
- Both are **same** as far as the variables **a** and **b** are concerned. But these notations differ when these are used with **assignment statements**.

  For example,
- **b = a++;** and **b = ++a;** are **different**
- In the first case (**b = a++;**) **a = 6** and **b = 5,** but in the later case both **a** and **b** will be **6**.
- The statement **b = a++;** is equivalent to the statement sequence

  b = a;

  a = a+1;
- The current value of **a** is used (**for assignment**) **first** and **then it is changed to a +1**

  (hence known as **use and change method**)

Contd…

- But the second statement **b = ++a;**
- is equivalent to the statement sequence

  a = a+1;

  b = a;
- Here, the value of **a** is **changed first** and the **new value is used for assigning**

  (hence known as **change and use method**).
- Similar is the case for **decrement operator…**

Sample Program (without using assignment)

```
#include<stdio.h>

int main()
{
      int a = 5, b=6;

      a++;
      printf("After Increment, a = %d\n",a); //now a will be 6
      b++;
      printf("After increment, b = %d\n",b); // now b will be 7

      return 0;
}
```

Sample Program (using assignment)

```c
#include <stdio.h>

int main()
{
    int a = 10, b = 10, result1, result2;

    result1 = ++a;  // Pre-increment: a becomes 11, result1 gets 11
    result2 = b++;  // Post-increment: result2 gets 10, then b becomes 11

    printf("After pre-increment, a = %d, result1 = %d\n", a, result1);
    printf("After post-increment, b = %d, result2 = %d\n", b, result2);

    return 0;
}
```